

```
[ ]: #DATA CLEANING

import pandas as pd
df=pd.read_excel("Data_Cleaning_50Plus_Records.xlsx")
print(df.head())

#2. Renaming columns
#Check current column Names
print(df.columns)

#Rename Columns
df.rename(columns={
    'Student ID':'Student_ID',
    'Student Name ':'Student_Name',
    ' Department':'Department'
},inplace=True)
print(df.columns)

#3. Changing Data Types
#check Data Types.
print(df.dtypes)

#Convert Student_ID to string
df['Student_ID']=df['Student_ID'].astype(str)

#Convert Age to Integer
#First handle missing values.
df['Age']=df['Age'].fillna(0) # this fill in null values as 0
df['Age']=df['Age'].astype(int)
#Check:
print(df.dtypes)

#4. Detecting Duplicate Records
#Find Duplicates
```

```
#4. Detecting Duplicate Records
```

```
#Find Duplicates
```

```
duplicates=df.duplicated()
```

```
print(duplicates)
```

```
[67]: #Count duplicates
      print(df.duplicated().sum())
```

```
0
```

```
[68]: #5. Dropping duplicate records
```

```
#Before
```

```
print(df.shape)
```

```
#Remove duplicates
```

```
df.drop_duplicates(inplace=True)
```

```
#After
```

```
print(df.shape)
```

```
(52, 6)
```

```
(52, 6)
```

```
[69]: print(df.columns)

Index(['Student_ID', 'Student_Name', 'Department', 'Age', 'Salary', 'Email'], dtype='object')
```

```
[70]: #6. String Operations
```

```
#String methods work only on text columns
```

```
#Remove Extra Spaces
```

```
df['Student_Name'] = df['Student_Name'].str.strip()
```

```
#Convert to uppercase
```

```
df['Student_Name'] = df['Student_Name'].str.upper()
```

```
print(df['Student_Name'].head())
```

```
# convert to title case
```

```
df['Student_Name'] = df['Student_Name'].str.title()
```

```
print(df['Student_Name'].head())
```

```
df['Student_Name'] = df['Student_Name'].str.strip()
#Convert to uppercase
df['Student_Name'] = df['Student_Name'].str.upper()
print(df['Student_Name'].head())
# convert to title case
df['Student_Name'] = df['Student_Name'].str.title()
print(df['Student_Name'].head())
```

```
0    STUDENT1
1    STUDENT2
2    STUDENT3
3    STUDENT4
4    STUDENT5
```

Name: Student_Name, dtype: object

```
0    Student1
1    Student2
2    Student3
3    Student4
4    Student5
```

Name: Student_Name, dtype: object

```
[72]: df['Department']=df['Department'].str.replace('CSE','Computer Science') #it is only for one val change
print(df['Department'].head())
```

```
df['Department']=df['Department'].replace({'CSE':'Computer Science',
                                             'ECE':'Electronics and communi eng',
                                             'EEE':'Electrical and electronic',
                                             'IT':'Information Tech',
                                             'MECH':'Mechanical'
})
print(df['Department'].head()) #for multi values use dict
```

```
0          IT
1          ECE
2          EEE
3          MECH
4    Computer Science
```

```
0          IT
1          ECE
2          EEE
3          MECH
4    Computer Science
Name: Department, dtype: object
0          Information Tech
1    Electronics and communi eng
2    Electrical and electronic
3          Mechanical
4          Computer Science
Name: Department, dtype: object
```

```
[73]: #String Length
df['Name_Length'] = df['Student_Name'].str.len()
print(df[['Student_Name', 'Name_Length']].head())
```

	Student_Name	Name_Length
0	Student1	8
1	Student2	8
2	Student3	8
3	Student4	8
4	Student5	8

```
[75]: #Extract First character
df['First_Letter'] = df['Student_Name'].str[0] #to replace single value want to use str and for multiple no need
print(df[['Student_Name', 'First_Letter']].head())
```

	Student_Name	First_Letter
0	Student1	S
1	Student2	S
2	Student3	S
3	Student4	S
4	Student5	S

[]:

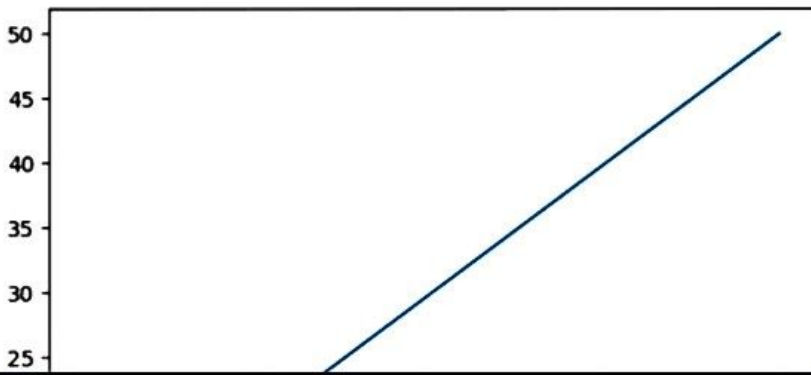
```
[1]: !pip install matplotlib
```

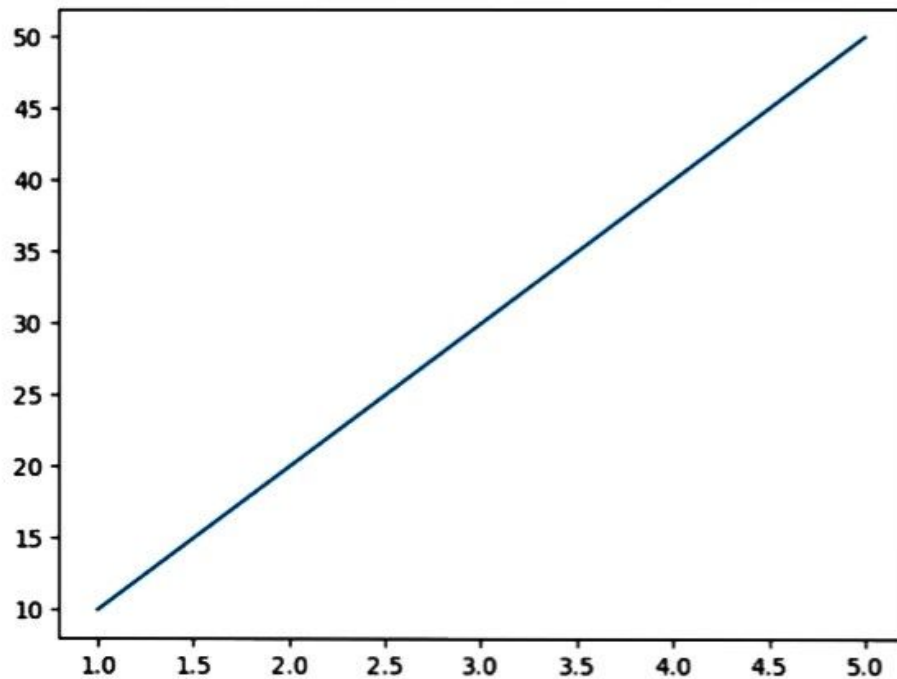


```
Requirement already satisfied: matplotlib in c:\users\harsh\anaconda3\lib\site-packages (3.9.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib) (1.4.4)
Requirement already satisfied: numpy>=1.23 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib) (1.26.4)
Requirement already satisfied: packaging>=20.0 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib) (24.1)
Requirement already satisfied: pillow>=8 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\harsh\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
```

```
[2]: import matplotlib.pyplot as plt
```

```
x = [1,2,3,4,5]
y = [10,20,30,40,50]
plt.plot(x,y)
plt.show()
```



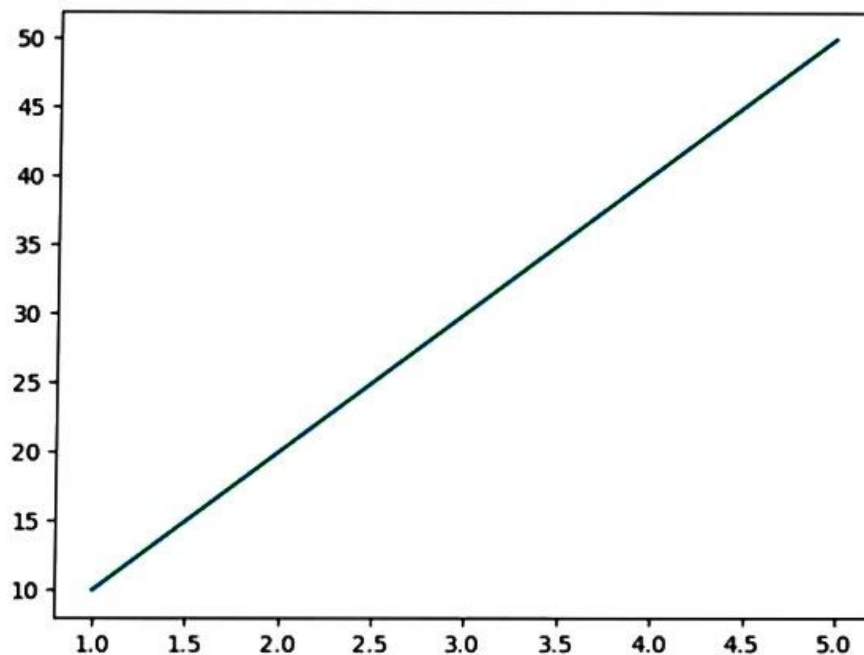


```
[8]: #Figure and Axes
#These are the most important concepts.
#Figure-The entire window.
#Axes-The area where the graph is drawn.
import matplotlib.pyplot as plt
fig,ax = plt.subplots()
ax.plot([1,2,3],[10,20,30])

#Adding a Title
plt.title("Monthly Sales")
#adding x and y labels
```

```
#Adding a Title
plt.title("Monthly Sales")
#adding x and y labels
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
#changing line colour
plt.plot(x,y,color='green')
#changing line style
plt.plot(x,y,linestyle='-.')
plt.show()
```





```
[11]: #Styles:
      #'-' Solid
      #'--' Dashed
      #':' Dotted
      #'-.' Dash Dot

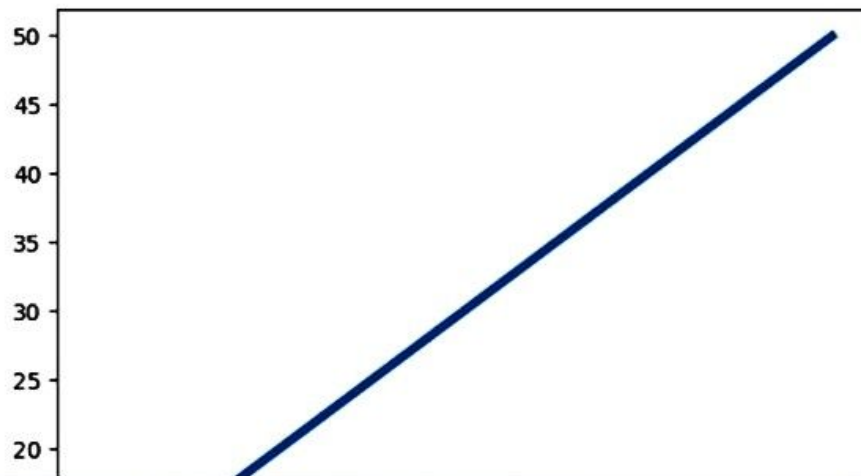
      #changing line width
      plt.plot(x,y,linewidth=4)
      plt.show()
      #Thicker line.
```

```
#changing line width
plt.plot(x,y,linewidth=4)
plt.show()
#Thicker line.
```

```
#Adding markers
plt.plot(x,y,marker='s')
plt.show()
```

```
#output:
#.....
#Marker types
```

```
#'o' Circle
#'s' Square
#'^' Triangle
#'*' Star
```



```
plt.show()
```

```
#output:
```

```
#.-----
```

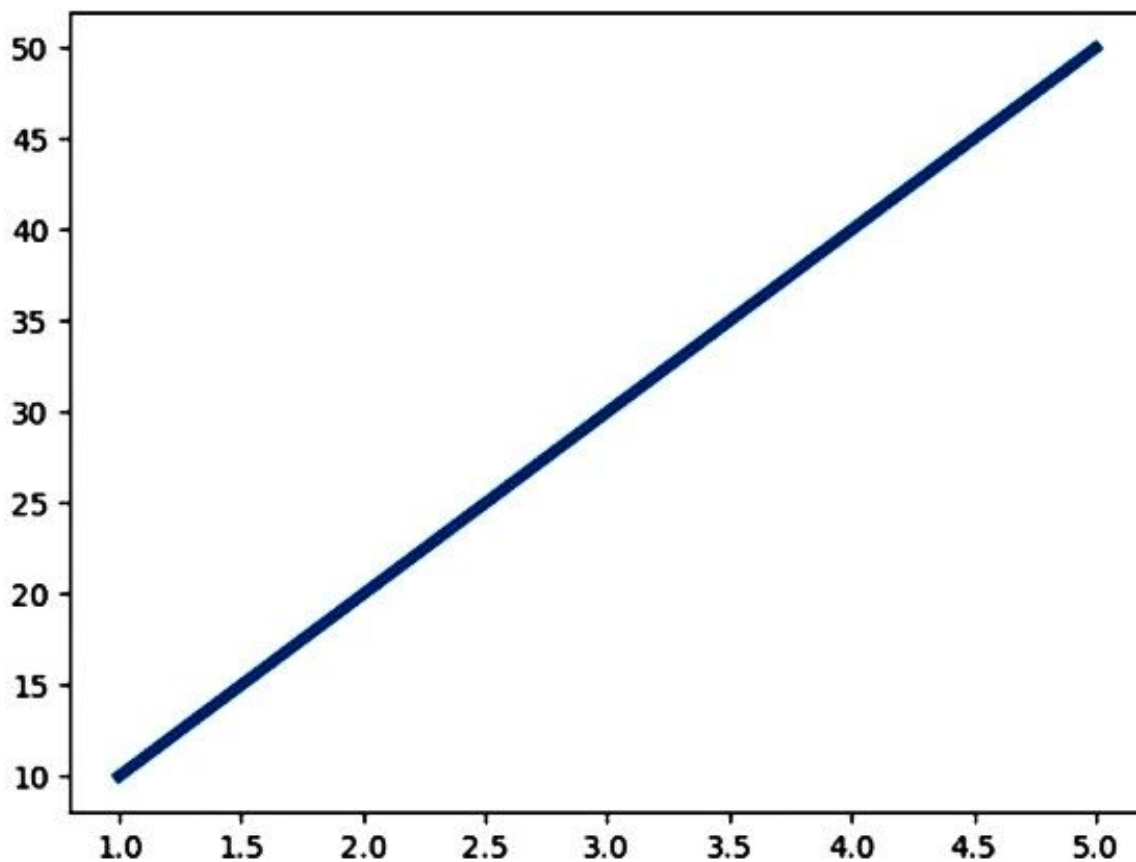
```
#Marker types
```

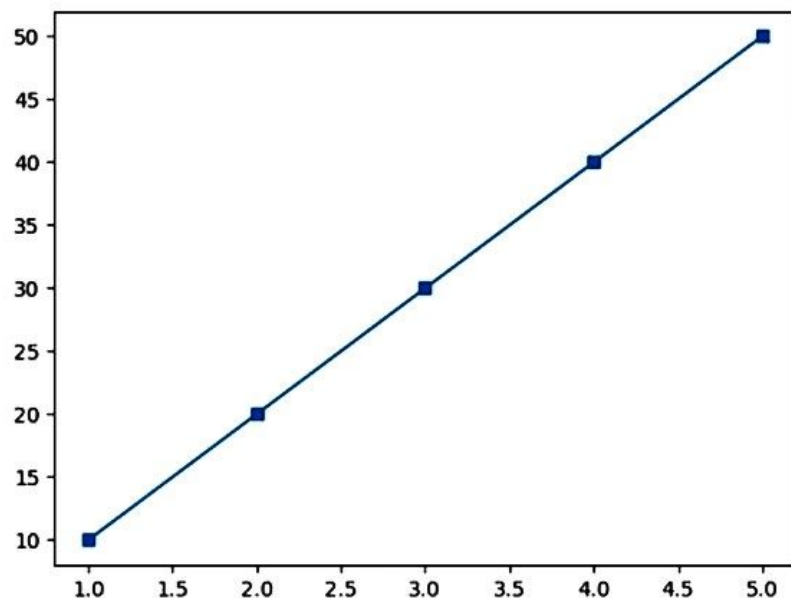
```
#'o' Circle
```

```
#'s' Square
```

```
#'^' Triangle
```

```
#'*' Star
```





```
[12]: !pip install seaborn
```

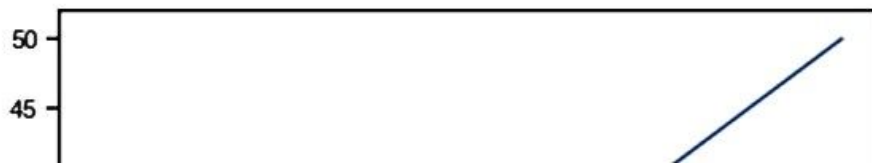
```
Requirement already satisfied: seaborn in c:\users\harsh\anaconda3\lib\site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\harsh\anaconda3\lib\site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in c:\users\harsh\anaconda3\lib\site-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\harsh\anaconda3\lib\site-packages (from seaborn) (3.9.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.4)
Requirement already satisfied: packaging>=20.0 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.1)
Requirement already satisfied: pillow>=8 in c:\users\harsh\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)
```

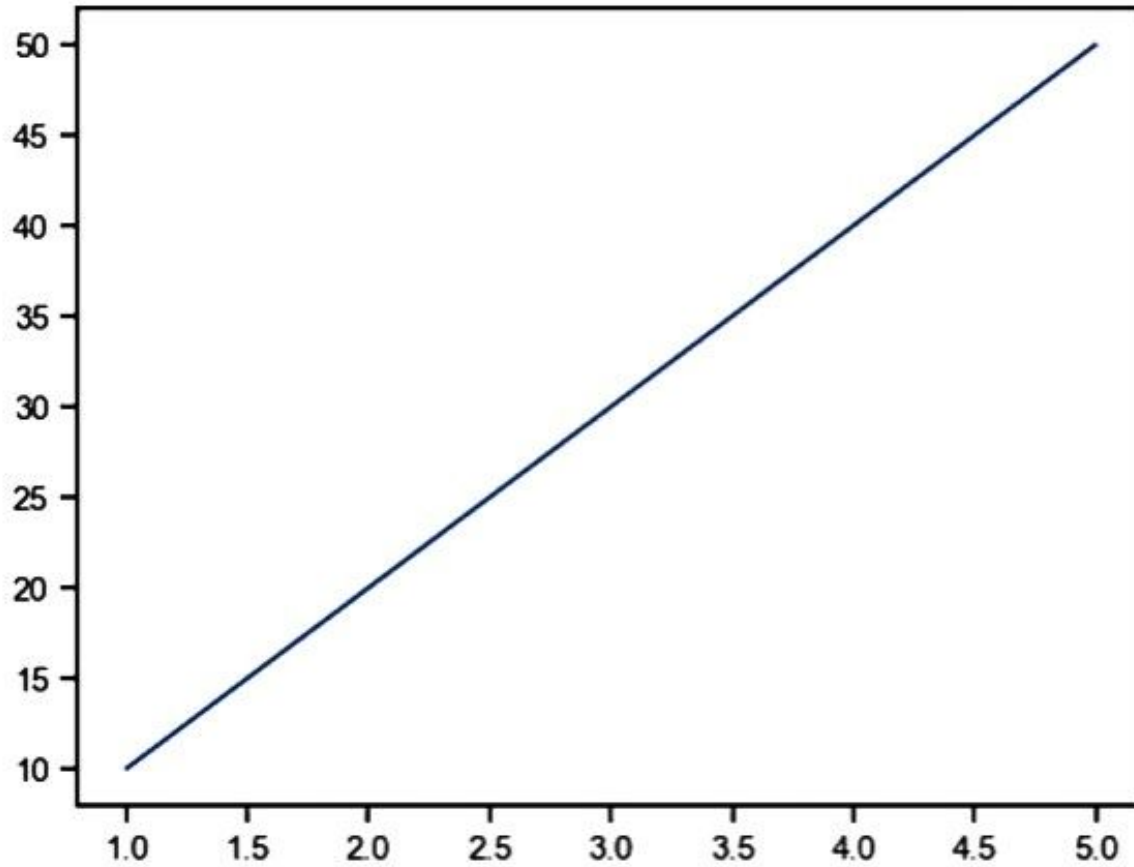
```
[27]: import seaborn as sns
import matplotlib.pyplot as plt
x=[1,2,3,4,5]
y=[10,20,30,40,50]

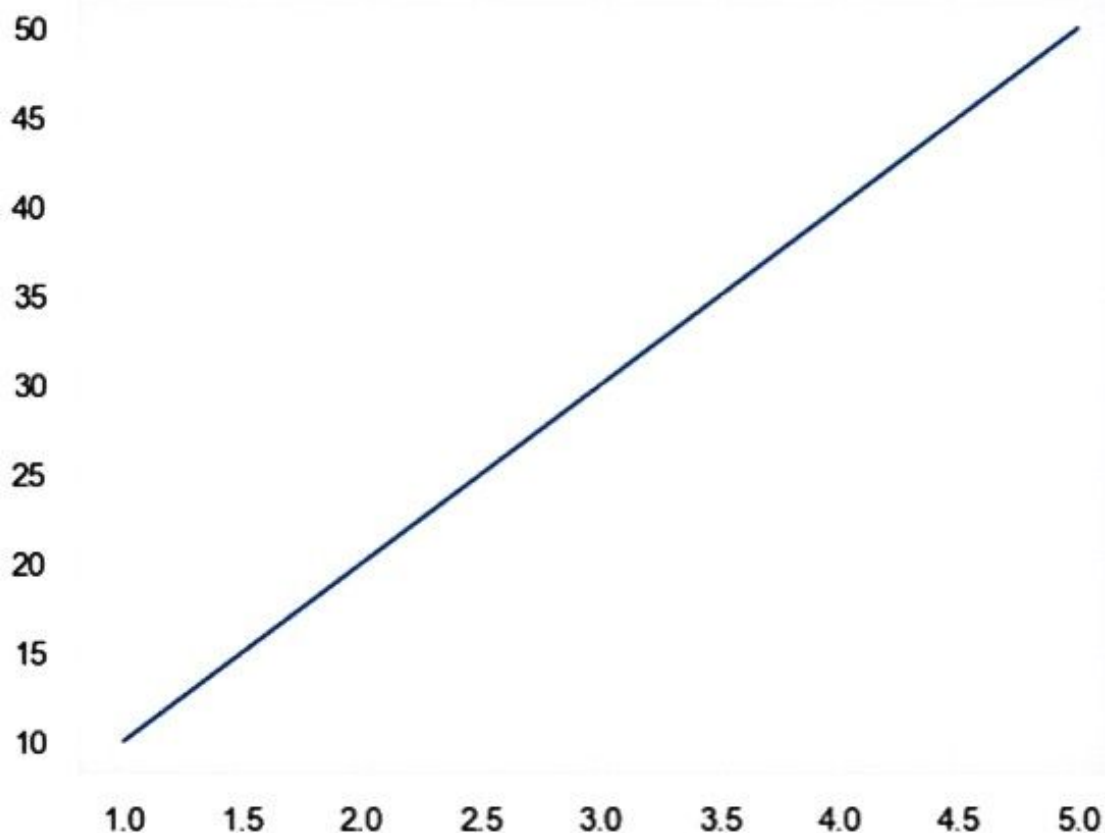
#themes in seaborn
#themes control the appearance of charts.
#default theme
sns.set_theme()
#dark Grid Styles
sns.set_style("ticks")
sns.lineplot(x=x,y=y)
plt.show()

sns.set_style("darkgrid")
sns.lineplot(x=x,y=y)
plt.show()

#available styles
#darkgrid
#whitegrid
#dark
#white
#ticks
```







```
[26]: #color palettes
      #A palette is a collection of colors.

      # view current palette
      sns.set_palette("bright") # the line will be bright compare to previous one
      sns.lineplot(
          x=[1,2,3,4],
          y=[10,20,30,40]
      )
```

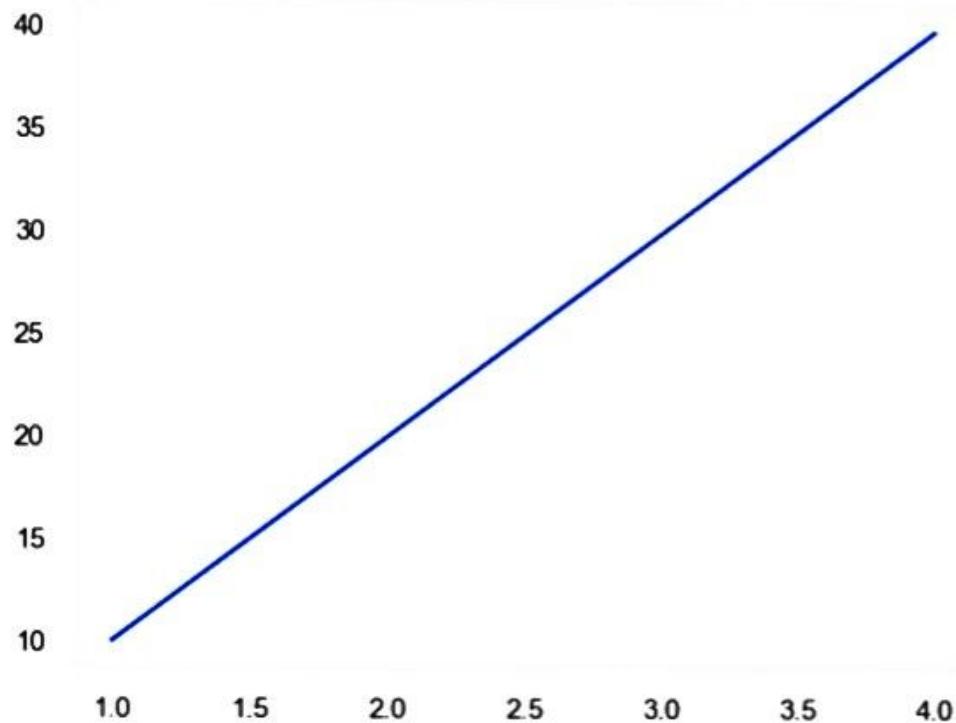
```
[26]: #color palettes
      #A palette is a collection of colors.

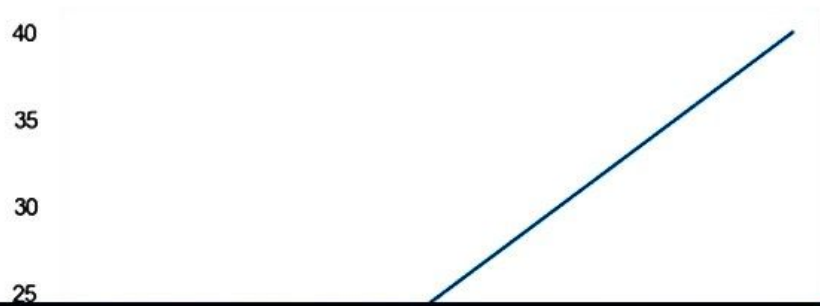
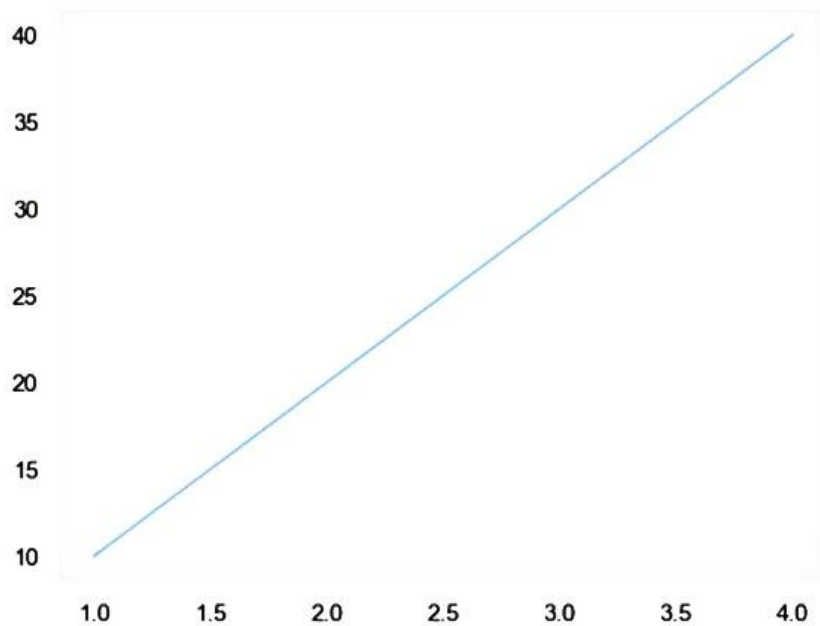
      # view current palette
      sns.set_palette("bright") # the line will be bright compare to previous one
      sns.lineplot(
          x=[1,2,3,4],
          y=[10,20,30,40]
      )
      plt.show()

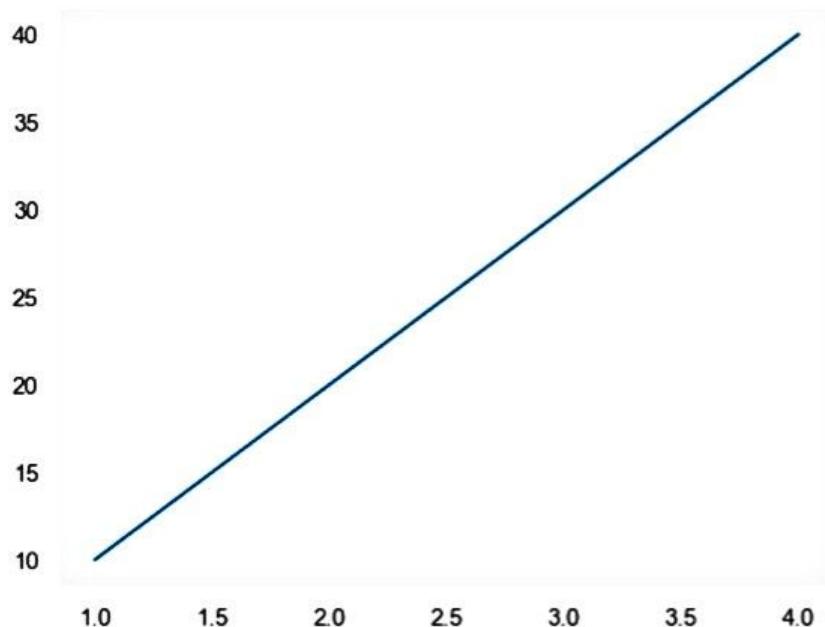
      sns.set_palette("pastel") # the line will be bright compare to previous one
      sns.lineplot(
          x=[1,2,3,4],
          y=[10,20,30,40]
      )
      plt.show()

      sns.set_palette("muted") # the line will be bright compare to previous one
      sns.lineplot(
          x=[1,2,3,4],
          y=[10,20,30,40]
      )
      plt.show()

      #available style
      #deep
      #bright
      #muted
      #pastel
      #dark
      #colorblind
```







```
[29]: #Built-in Dataset support
      #one of seaborn's biggest advantages is built-in dataset.
      #View available datasets
      import seaborn as sns
      print(sns.get_dataset_names())
```

```
['anagrams', 'anscombe', 'attention', 'brain_networks', 'car_crashes', 'diamonds', 'dots', 'dowjones', 'exercise', 'flights', 'fmri',
'healthexp', 'iris', 'mpg', 'penguins', 'planets', 'seaice', 'taxi', 'tips', 'titanic']
```

```
[30]: #Load Titanic Dataset
```

[30]: #Load Titanic Dataset

```
import seaborn as sns
df=sns.load_dataset("titanic")
print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class \
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

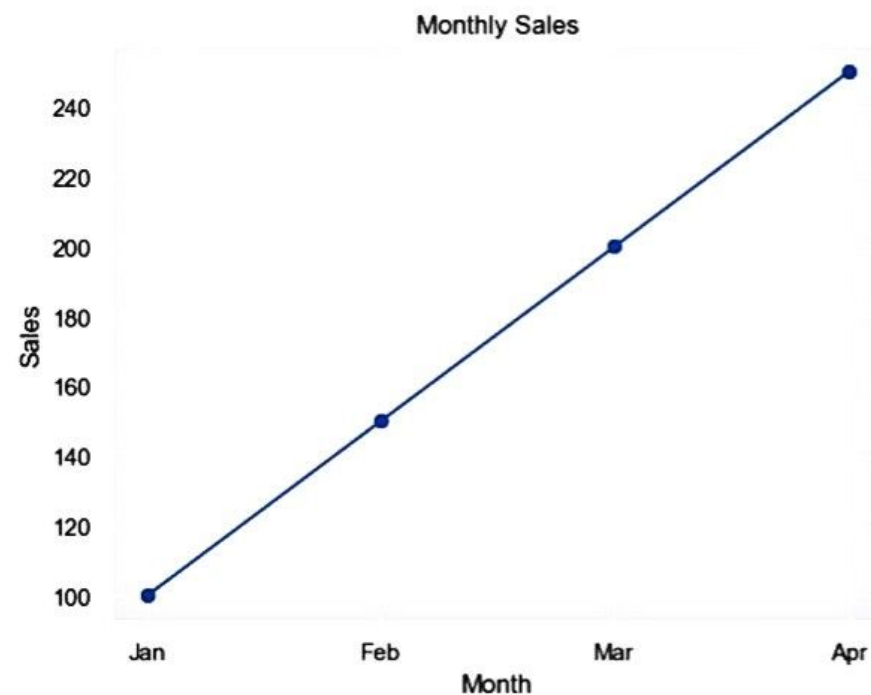
	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

[32]: #Line chart

```
#what ia a line chart?
# A line chart connects data points with lines.
# When to use?
#Showing trends over time
#Examples:
#-Monthly Sales
#-Tempertaures changes
#-stock price
#-Student performance over semesters
```

```
import matplotlib.pyplot as plt
months=['Jan','Feb','Mar','Apr']
sales=[100,150,200,250]
plt.plot(months,sales,marker='o')
plt.title("Monthly Sales")
plt.xlabel("Month")
```

```
import matplotlib.pyplot as plt
months=['Jan','Feb','Mar','Apr']
sales=[100,150,200,250]
plt.plot(months,sales,marker='o')
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```



```
[33]: #Bar chart
#What is a bar chart?
#A bar chart uses rectangular bars to compare values.
#When to use

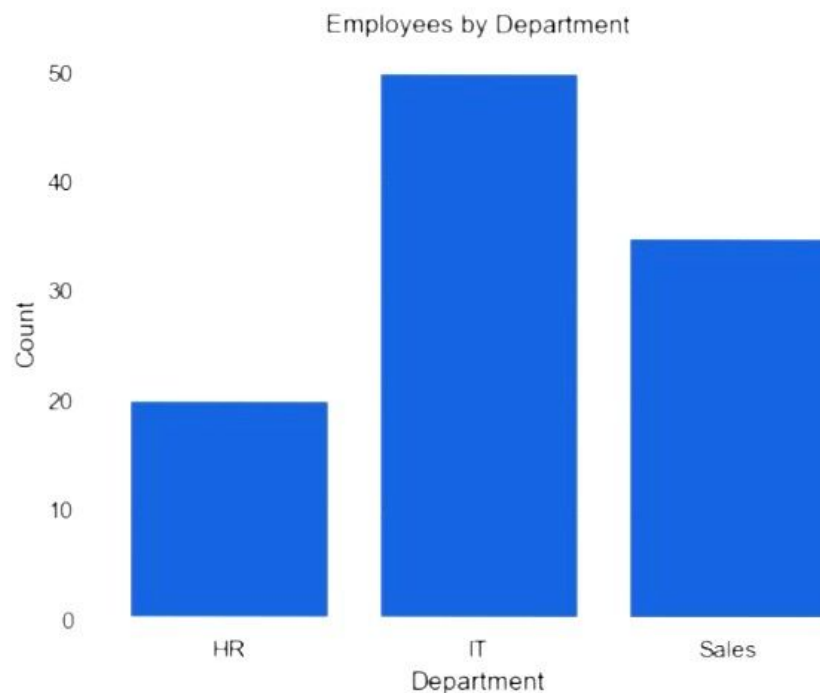
#Comparing categories

#Examples
#-Dept wise employees
#-Product sales
#-Student sales
#Population by city

import matplotlib.pyplot as plt
dept=['HR','IT','Sales']
employees=[20,50,35]
plt.bar(dept,employees)
plt.title("Employees by Department")
plt.xlabel("Department")
plt.ylabel("Count")
plt.show()
```



`plt.show()`

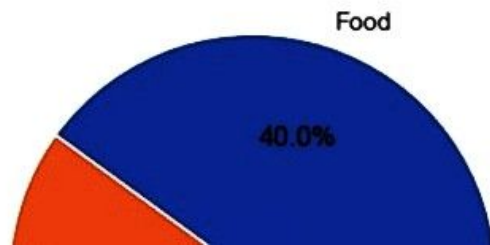


```
#Pie chart
#what is a pie chart
#A pie chart shows a parts of a whole
#When to use?
# showing percentages
# Examples
#-Budget allocations
```

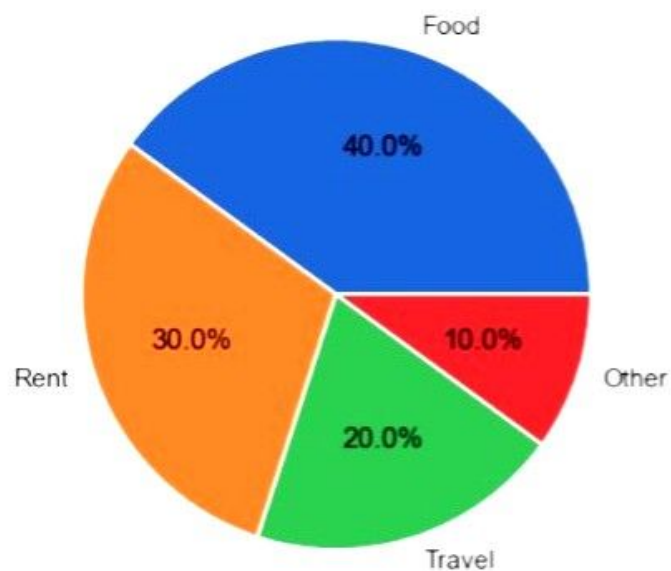
```
[34]: #Pie chart
#What is a pie chart
#A pie chart shows a parts of a whole
#When to use?
# showing percentages
# Examples
#-Budget allocations
#-Market share
#-Survey results
#-Expenses distributions

import matplotlib.pyplot as plt
labels=['Food','Rent','Travel','Other']
values=[40,30,20,10]
plt.pie(
    values,
    labels=labels,
    autopct='%1.1f%%'
)
plt.title("Monthly Expenses")
plt.show()
```

Monthly Expenses



Monthly Expenses



#Scatter chart

#What is a scatter plot?

#A scatter plot uses dots to represent data points

#When to use

#Finding relationships between two variables

#Examples

#-Height vs weight

```
[36]: #Scatter chart
#What is a scatter plot?

#A scatter plot uses dots to represent data points

#When to use

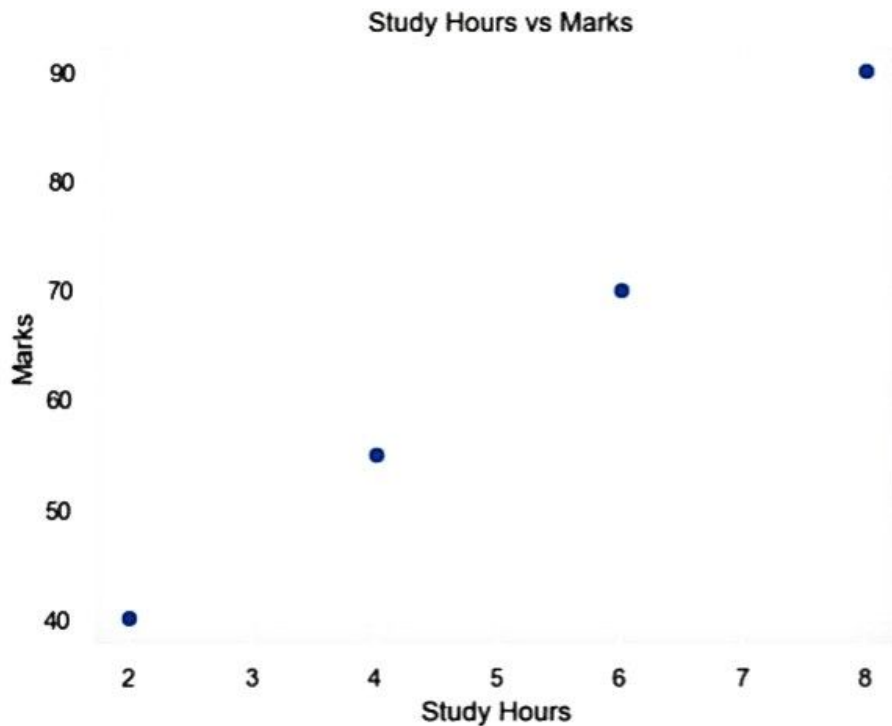
#Finding relationships between two variables
#Examples
#-Height vs weight
#-Study Hours vs Marks
#-Experience vs Salary

import matplotlib.pyplot as plt
hours=[2,4,6,8]
marks=[40,55,70,90]
plt.scatter(hours,marks)
plt.title("Study Hours vs Marks")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.show()
```

Study Hours vs Marks

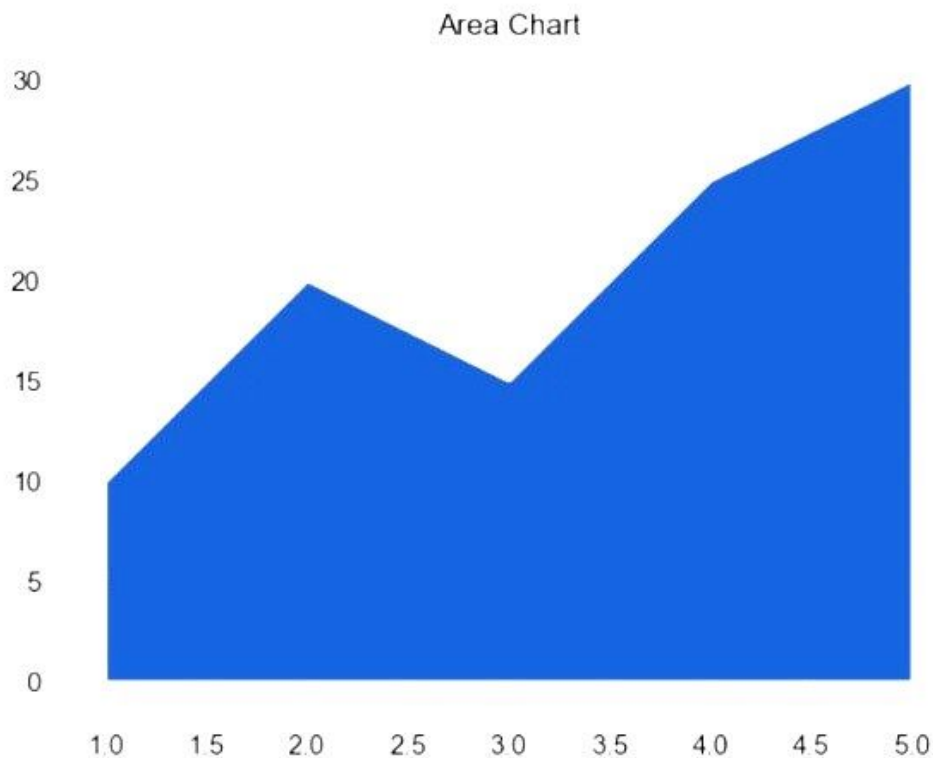
90

80



```
[38]: #Area chart
#Purpose: Shows cumulative totals over time.
import matplotlib.pyplot as plt
x=[1,2,3,4,5]
y=[10,20,15,25,30]
plt.fill_between(x,y)
plt.title("Area Chart")
```

```
#Area chart  
#Purpose: Shows cumulative totals over time.  
import matplotlib.pyplot as plt  
x=[1,2,3,4,5]  
y=[10,20,15,25,30]  
plt.fill_between(x,y)  
plt.title("Area Chart")  
plt.show()
```



```
[39]: ## BOX PLOT
# A Box plot is used to visualize the distribution of numerical data identify outliers.

#components of a box plot
#Min---Q1 median Q3---Max
#   /               /
#Lower whisker    upper whisker

#Minimum: smallest value(excluding outliers)
#Q1 (25%): First quartile
#Median (50%: Middle value
#Q3 (75%): Third quartile
#Maximum: Largest value(excluding outliers)
#Outliers: Values far from the rest of the data,shown as individual points.

import matplotlib.pyplot as plt
salary= [25000,28000,30000,32000,35000,
          37000,40000,42000,45000,100000]
plt.boxplot(salary)
plt.title("Salary Distribution")
plt.ylabel("Salary")
plt.show()
```

Salary Distribution

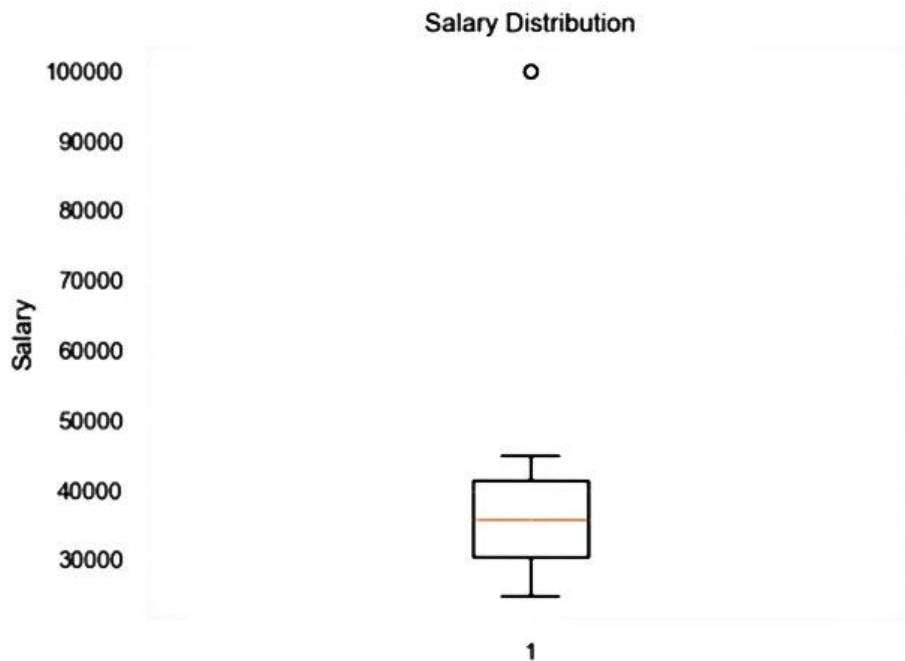
100000

○

90000

80000

70000



```
[43]: #Heat Map
#What is Heat map?
#A heat map uses colors to represent values in a table or matrix.

#In data analysis,heatmaps are commonly used to visualize correlations between numerical columns.

#Correlation values
# value meaning
# +1 Strong positive relationships
```

1

```
[43]: #Heat Map
#What is Heat map?
#A heat map uses colors to represent values in a table or matrix.

#In data analysis,heatmaps are commonly used to visualize correlations between numerical columns.

#Correlation values
# value meaning
# +1 Strong positive relationships
# 0 No
# -1 Strong negative relationship

#correlation Heatmap with seaborn
import seaborn as sns
import matplotlib.pyplot as plt

#Load dataset
tips=sns.load_dataset("tips")

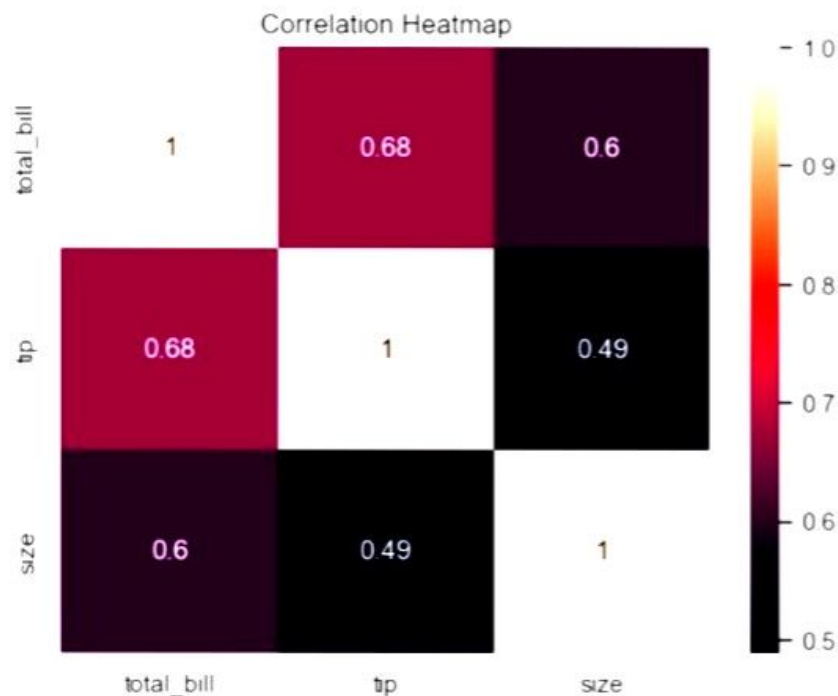
#Correlation matrix
corr=tips.corr(numeric_only=True)

#Heatmap
sns.heatmap(corr,annot=True)

plt.title("Correlation Heatmap")
plt.show()
```

Correlation Heatmap

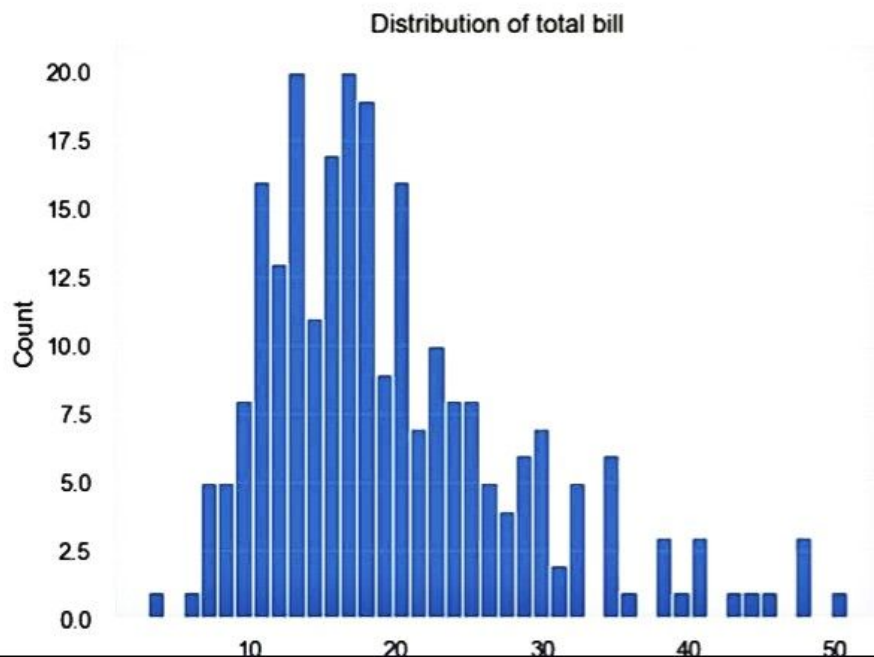
- 1.0



```
#Histogram
sns.histplot(
    data=tips,
    x="total_bill",
    bins=40
)
plt.title("Distribution of total bill")
plt.show()
```

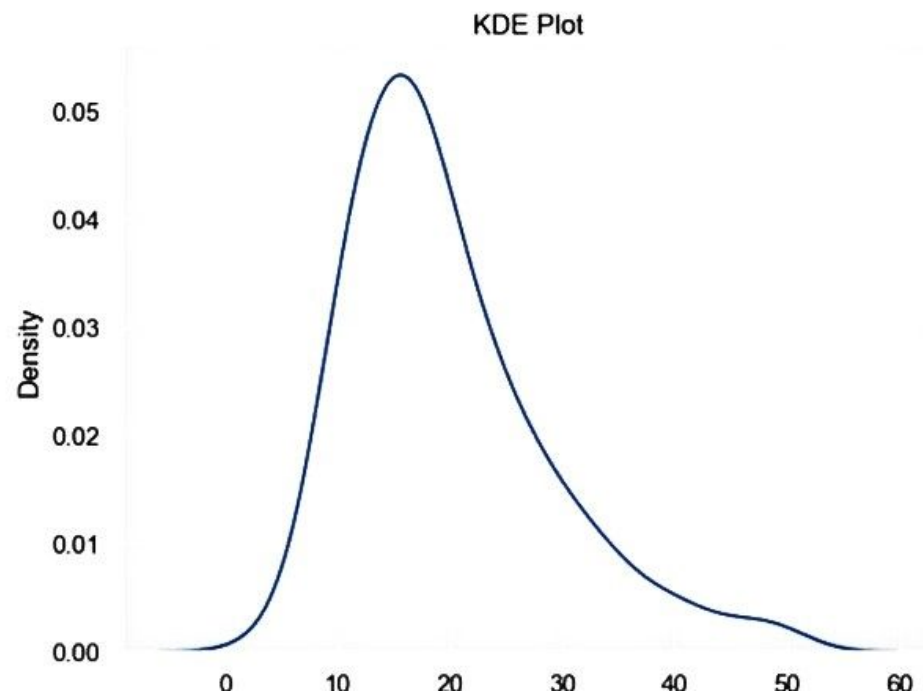
```
[51]: #Histogram
sns.histplot(
    data=tips,
    x="total_bill",
    bins=40
)
plt.title("Distribution of total bill")
plt.show()

#sns.histplot()
#Creates histogram
```



#understanding data distribution identifying peaks computing distribution statistical analysis.

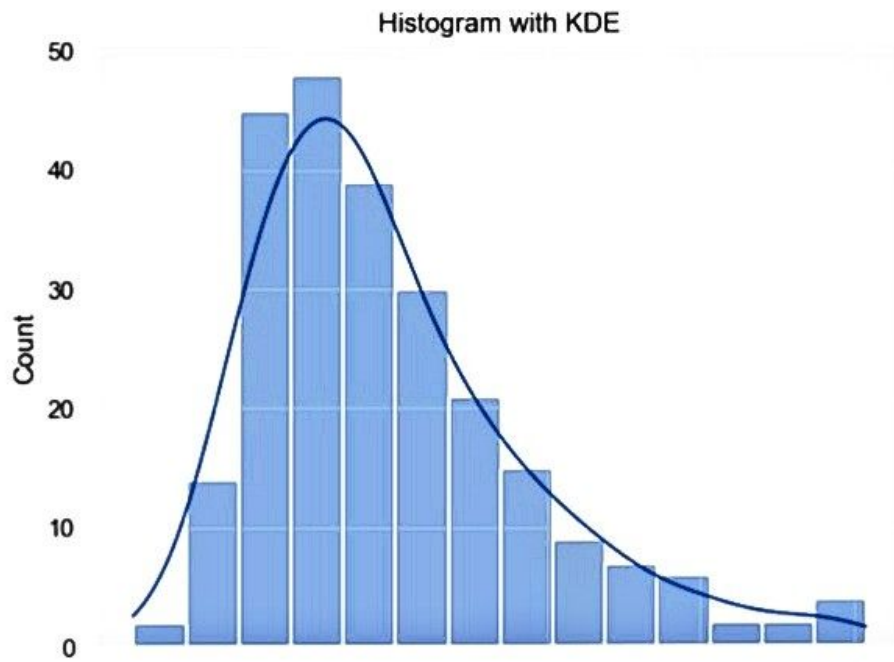
```
sns.kdeplot(
    data=tips,
    x="total_bill",
)
plt.title("KDE Plot")
plt.show()
```



```
[54]: sns.histplot(data=tips, x="total_bill", kde=True)
```

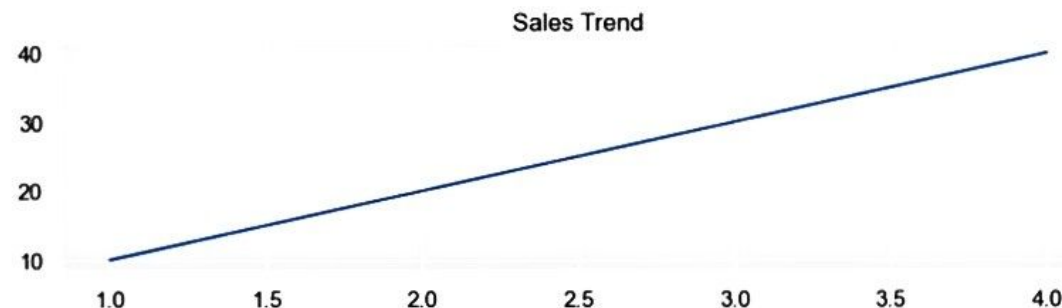
#most commonly used in data science

```
sns.histplot(  
    data=tips,  
    x="total_bill",  
    kde=True  
)  
plt.title("Histogram with KDE")  
plt.show()
```



```
[56]: #1. Figure Sizing What is figure size
      #Figure size controls the width and height of the chart
```

```
import matplotlib.pyplot as plt
plt.figure(figsize=(9,2))
plt.plot([1,2,3,4],[10,20,30,40])
plt.title("Sales Trend")
plt.show()
```



```
[57]: #Subplots
      #what is subplots
```

```
#2 charts
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,4))

# First Plot
```

```
[57]: #Subplots
      #what is subplots

      #2 charts

      import matplotlib.pyplot as plt

      plt.figure(figsize=(8,4))

      # First Plot

      plt.subplot(1,2,2)

      plt.plot([1,2,3], [10,20,30])

      plt.title("Line Chart")

      # Second Plot

      plt.subplot(1,2,1)

      plt.bar(['A','B','C'], [5,8,6])

      plt.title("Bar Chart")

      plt.show()
```

Bar Chart



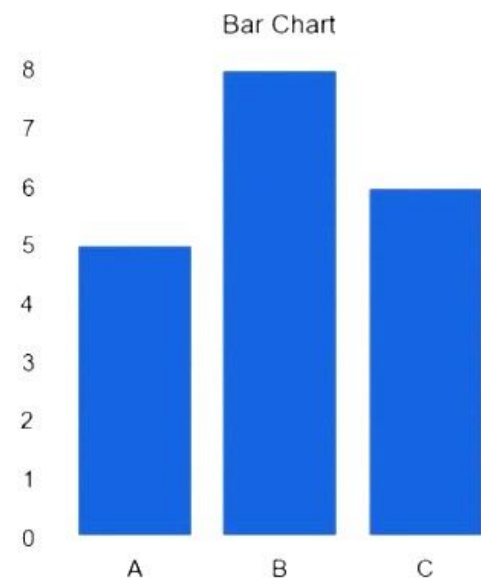
Line Chart



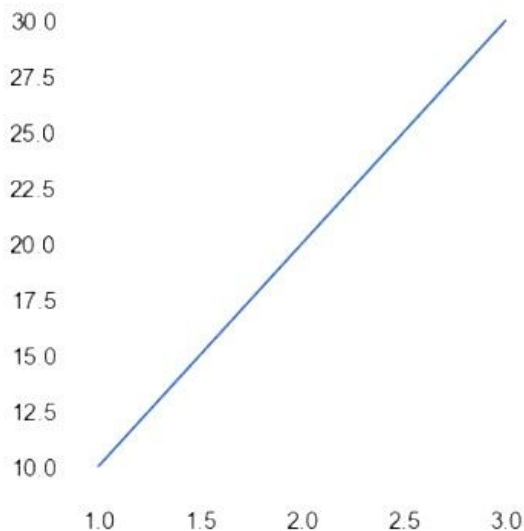
```
plt.bar(['A','B','C'], [5,8,6])
```

```
plt.title("Bar Chart")
```

```
plt.show()
```



Line Chart



#Example: Four Charts

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10,8))
```

```
plt.subplot(2,2,1)
```

```
[60]: #Example: Four Charts

import matplotlib.pyplot as plt

plt.figure(figsize=(10,8))

plt.subplot(2,2,1)

plt.plot([1,2,3], [10,20,30])

plt.title("Line")

plt.subplot(2,2,2)

plt.bar(['A','B','C'], [5,8,6])

plt.title("Bar")

plt.subplot(2,2,3)

plt.scatter([1,2,3], [4,8,6])

plt.title("Scatter")

plt.subplot(2,2,4)

plt.pie([30,40,30], labels=['A','B','C'])

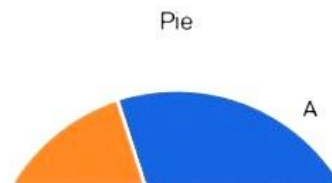
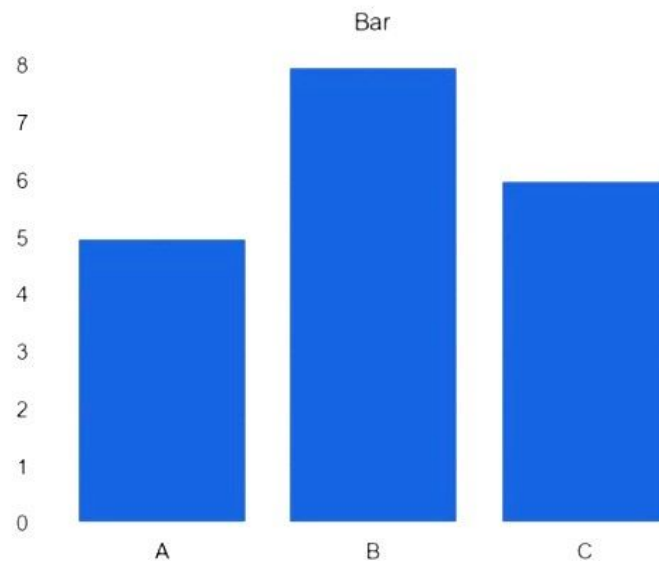
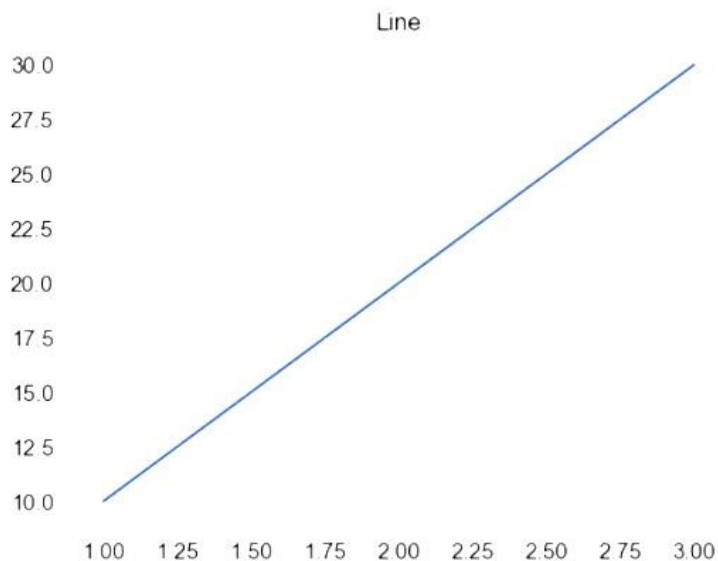
plt.title("Pie")
plt.tight_layout()
plt.show()
```

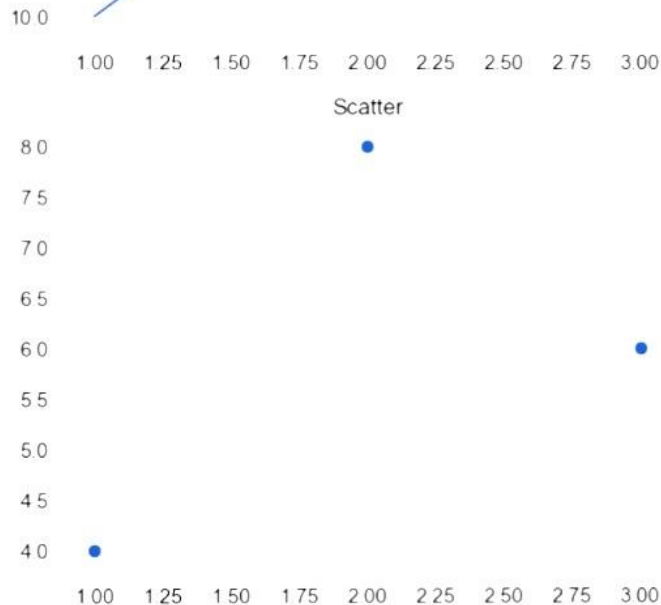
Line

Bar

```
plt.pie([30,40,30], labels= 'A','B','C')
```

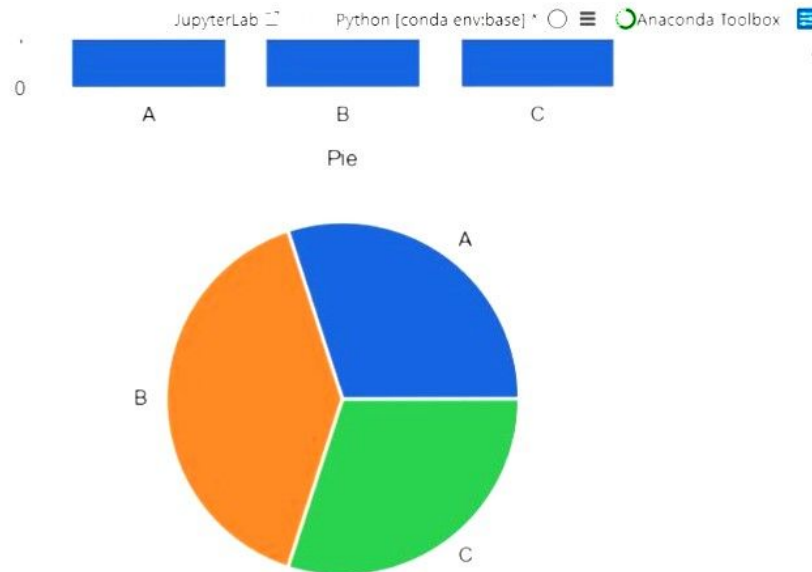
```
plt.title("Pie")
plt.tight_layout()
plt.show()
```



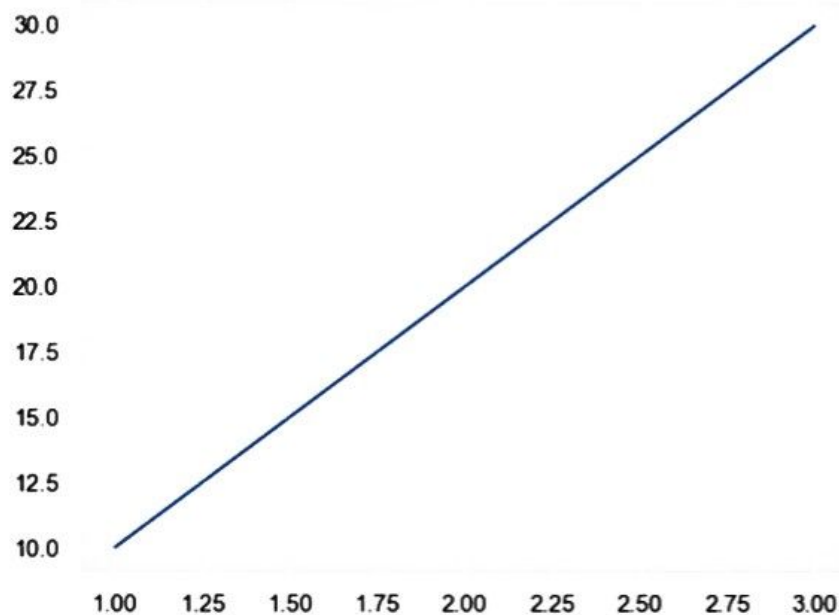


```
#plt.savefig() used to save charts as image files
import matplotlib.pyplot as plt
plt.plot( 1,2,3, |10,20,30 )
plt.title("Sales Trend")
plt.savefig("sales_chart.png")
plt.show()
```

Sales Trend



Sales Trend



```
[63]: #save with high quality
plt.savefig(
    "chart.png",
    dpi=300
)
```

<Figure size 640x480 with 0 Axes>

```
[ ]:
```